

Zadanie domowe Podstawy Programowania 2025/26

Ogólne wytyczne pisania programów

Program musi być zgodny z dobrymi praktykami programistycznymi i mieć czytelny kod. Gra powinna być łatwa w obsłudze, działać płynnie bez błędów, lagów i awarii. Oceniana będzie także estetyka i poprawność kodu. Uchybienia mogą spowodować pomniejszeniem liczby punktów. Program musi być stworzony z wykorzystaniem podprogramów. Nie można stosować zmiennych globalnych, można stosować stałe globalne (np. wymiary planszy, definicje pionów itp.)

1. Samodzielność:

- a. zadanie musi zostać napisane samodzielnie,
- b. nie można dzielić się kodem,
- c. nie można pokazywać swojego kodu innym,
- d. nie można pisać w grupach,
- e. nie można umieszczać fragmentów cudzego kodu w swoim programie,
- f. nie można przepisywać kodu z książek, z Internetu, genAI i innych.

2. Metadane w kodzie - komentarz nagłówkowy (1 linia kodu) powinien zawierać dane Autora: Imię, nazwisko, nr. Indeksu, kierunek studiów, grupę laboratoryjną, tytuł zadania. Funkcje (podprogramy) powinny mieć w komentarzu krótki opis do czego służą.

3. Oceniany jest kod wysłany w terminie na [STOS](#), po upływie terminu nie ma możliwości nadesłania zadania. Przed terminem końcowym, można wielokrotnie wysyłać kod, również wersje pośrednie – zachęcamy. Liczy się ostatnia nadesłana wersja.

4. Program musi być stworzony z wykorzystaniem podprogramów.

5. Nie można stosować zmiennych globalnych, można stosować stałe globalne (np. wymiary planszy, definicje itp.).

6. Należy postępy pracy dokumentować na serwisie [GITPG](#), wrzucając systematycznie nowsze wersje kodu, wraz z tą ostateczną.

7. Należy stworzyć krótki raport w pliku PDF, w formie tabelki. Wiersze tabelki odpowiadają punktom zadania (od 1 do 3.4), jak w tabeli „Punktacja” na końcu niniejszej instrukcji. W pierwszej kolumnie tabelki pokaż fragment kodu (krótki, ok ~20 linii, np. funkcja) realizujący dany fragment zadania (jeśli jest bardzo rozbudowany, w wielu miejscach kodu, to wybierz najważniejszy fragment), oraz w drugiej kolumnie tabelki zamieść fragment wyniku działania programu (np. zrzut ekranu) obrazujący to działanie. Raport wrzucamy na GIT/MOJAPG, w terminie oddania zadania. Brak raportu jest równoznaczny z brakiem oddania zadania. Zalecane jest systematyczne uzupełnianie raportu, podobnie jak praca z kodem w GIT.

Zadanie

Napisz program w C++, który będzie tworzył grafikę za pomocą obsługi plików .SVG. Pliki SVG (Scalable Vector Graphic) to plik tekstowy grafiki wektorowej, który za pomocą języka XML (Extensible Markup Language) opisuje grafikę wektorową jako zestaw kształtów (np. prostokątów, linii, okręgów itp.). Plik ten może być wyświetlany np. w przeglądarce internetowej.

Twoim zadaniem jest napisanie programu, który potrafi pisać treść plików tekstowych zgodnie z zasadami plików SVG aby móc uruchomić je jako grafikę w przeglądarce. Treść wygenerowanego pliku będzie zależeć od działania użytkownika programu. W podstawowej wersji, program służy do gry w kółko i krzyżyk a plansza gry wyświetlana jest w konsoli za pomocą znaków ASCII, oraz tworzy plik SVG z aktualnym stanem planszy gry, wyrysowanym za pomocą poleceń właściwych dla plików SVG, jak np. koło o środku w punkcie (125,125) i promieniu $r=75$: `<circle cx="125" cy="125" r="75"/>`, prostokąt zaczynający się w punkcie (25,25) o wysokości 200 i szerokości 200: `<rect x="25" y="25" width="200" height="200"/>` i inne. Sprawdź dokumentację plików SVG : [W3](#), [wiki online SVG editor](#). W dalszej części program umożliwia grę w warcaby.

Zadanie posiada listę wymagań, od których wykonania zależy możliwa do uzyskania liczba punktów. Za cały projekt można uzyskać 20 punktów. Wymagania i punktacja zamieszczone są skrótowo w tabeli na końcu dokumentu. Poniżej opisane są szczegółowe wymagania poszczególnych części zadania - etapów. Kolejne etapy rozbudowują funkcjonalność programu, a NIE SĄ osobnymi programami.

!!! Zadania powinny być wykonywane po kolei, tzn. program jest rozbudowywany do danego momentu. Wymogi z * można pominąć i przejść do dalszego podpunktu, najwyżej nie uzyskasz punktów za ten wymóg, ale możesz realizować dalsze.

ETAP 1 - Podstawowa obsługa SVG.

1.1. Program generuje pusty plik SVG którego kod XML pozytywnie się waliduje (za pomocą W3C Validator) oraz jest uruchamiany przez przeglądarkę.

1.2. Plik SVG posiada 3 elementy:

- jeden okrąg, jeden prostokąt oraz jedną linię, (`<circle>`, `<rect>`, `<line>`),
- koordynaty XY, rozmiar, oraz kolor (wypełnienia/linii).

Na tym etapie grafika jest dowolna, a rozmieszczenie elementów oraz ich parametry podawane są przez użytkownika. Zadbaj o to, żeby element na siebie nie nachodziły, a jeśli tak to poproś użytkownika o poprawne dane.

1.3. Program powinien posiadać menu wyboru z opcjami:

- 1) generowanie podstawowego pliku SVG (1.1),
- 2) generowanie zaawansowanego pliku SVG (1.2),
- 3) gra w kółko i krzyżyk (Etap 2),
- 4) gra w warcaby (Etap 3),
- 5) oraz (H) pomoc – opis w jaki sposób grać w grę z etapu 2 oraz etapu 3 (np. jakie dane użytkownik wprowadza podczas gry, jakie klawisze służą do wykonywania określonych działań itp.),
- 6) a także (X) – wyjście z programu.

ETAP 2 - Gra w kółko i krzyżyk.

2.1. Napisz w C++ grę w kółko i krzyżyk z grafiką w ASCII wypisywaną na ekran. Program powinien posiadać w menu dodatkową opcję (3) gra w kółko i krzyżyk.

- w tej grze plansza gry to 3x3 pola, gdzie gracze naprzemiennie stawiają O lub X w jedno z tych 9-ciu pól. Warunkiem wygranej jest ułożenie 3 znaków jednego gracza w linii pionowej, poziomej lub ukośnej.
- gracze stawiają znaki naprzemiennie, najpierw O, potem X.
- grają dwaj gracze-ludzie, wykorzystując standardowe wejście i wyjście. Sposób podawania danych, na którym polu ma być postawiony znak, opracowuje programista.
- sprawdzaj warunki wygranej, gra kończy się jeśli jeden z graczy wygra lub nie można postawić znaku.

2.2. Dodatkowo, gra na starcie generuje plik SVG z pustą planszą (kwadrat z 9 oddzielonych od siebie pól, np. kreskami), a następnie po każdym ruchu stawia O lub X w odpowiednie pole za pomocą dopisania polecenia (np. `<circle>` lub `<rect>` z odpowiednimi parametrami) do treści pliku tekstowego SVG. Plik z danym etapem gry może być nadpisywany. Możesz grać w swoim programie i w przeglądarce obok wyświetlać plik SVG i odświeżać go po każdym kroku w grze. Uwaga: plik SVG zmienia zawartość w trakcie działania twojego programu, należy więc odświeżać go w przeglądarce co tura (np. klawiszem f5).

2.3. Dodaj możliwość gry z graczem (człowiek) lub z komputerem (PC), dodając do menu nową opcję (4) Kółko i krzyżyk z komputerem. Gracz PC ma prostą logikę:

- (1) pierwszy krok - jeśli gra O to stawia na środek, jeśli nie to gdziekolwiek,
- (2) jeśli przeciwnik ma dwa swoje znaki w linii to blokuje,
- (3) stawia swój znak przy innym swoim znaku,
- (4) jeśli nie ma opcji na żadne z powyższych, to stawia losowo.
- (?) jeśli nie zrobisz logiki to zrób samo stawianie losowe (mniej punktów, ale możesz się skupić na innych etapach zadania).

2.4. Dodaj licznik czasu gry dla użytkowników, kończ grę po przekroczeniu czasu gry (np. 1 minuta na gracza).

ETAP 3 – gra w Warcaby.

3.1. Dodaj do menu programu funkcję (5) gra w warcaby. Program umożliwia grę w warcaby. Szczegółowe wymagania na grę w warcaby [link](#) :

- Plansza 8x8, szachownica (odmienne kolory tła),
- Gracze mają po 12 pionów ułożonych w 3 rzędy na czarnych polach, zaczyna biały
- Wyświetlanie gry w konsoli znakami ASCII,
- Sprawdzanie warunków ruchu,
- Bicie jest obowiązkowe!
- Sprawdzanie warunków wygranej,
- Naprzemienna gra graczy,

3.2. Dodaj generację grafiki za pomocą plików .SVG, podobnie jak poprzednio w kółko i krzyżyk. Wyświetlanie planszy szachownicy w .SVG wraz z ustawieniem figur jako bardziej złożonych obrazków w odpowiednim polu. Pełna grywalna gra w warcaby gracz vs gracz z grafiką w SVG.

3.3. Obsługa zamiany pionka w „damkę” – inny typ pionka i inne warunki ruchów damki niż pionka – „damka” porusza się tylko po przekątnej. Inna grafika „damki” niż pionka.

3.4. Zapis historii stanu gry do pliku i odczyt stanu gry z pliku i kontynuacja gry.

(+) Systematyczna praca z GIT, demonstracja w politechnicznym systemie GITPG, przynajmniej 4 zatwierdzeń (commit'ów) w odstępach co najmniej 3 dni (przed terminem oddania) zawierających realizację różnych podpunktów, w tym ostatni.

(++) Komentarze do funkcji.

!!! Nie zapomnij o RAPORCIE !!!

Punktacja:

1.1	Program tworzy poprawny pusty plik SVG.	0,5 pkt
1.2	Plik zawiera 3 kształty podstawowe (<circle>, <rect>, <line>), o zadanych przez użytkownika współrzędnych, kolorach i rozmiarze.	1 pkt
1.3	Opracowanie menu programu.	0,5 pkt
2.0	Wyświetlanie planszy do kółka i krzyżyka w konsoli z ASCII.	0,5 pkt
2.1	Działający mechanizmy gry w kółko i krzyżyk (w konsoli) gracz vs gracz – poprawne stawianie znaków i warunek wygranej.	4 pkt
2.2	Generowanie planszy gry, w pliku .SVG i aktualizacja krok po kroku w trakcie gry (można odświeżać plik w przeglądarce żeby widzieć grę „na żywo”).	3 pkt
2.3*	Mechanizm rozbudowany o grę gracz vs. PC z podstawowym algorytmem gry PC: wpierw zablokować by nie przegrać, potem losowo stawiać znak obok swojego, zgodnie z zasadami gry.	2 pkt
2.4*	Licznik czasu graczy i przegrana po przekroczeniu czasu (np. 30 s na gracza)	1 pkt
3.1	Gra w warcaby z podstawowym zakresem ruchów, 1gracz vs 1 gracz, wyświetlanie gry w konsoli znakami ASCII, sprawdzanie warunków ruchu. Sprawdzanie warunków wygranej. Naprzemienna gra graczy.	2 pkt
3.2	Wyświetlanie planszy szachownicy w .SVG wraz z ustawieniem figur jako bardziej złożonych obrazków w odpowiednim polu.	2 pkt
3.3*	Obsługa zamiany pionka w „damkę” – inny typ pionka i inne warunki ruchów damki niż pionka. Inna grafika „damki”.	1 pkt
3.4*	Zapis historii stanu gry do pliku i odczyt stanu gry z pliku i kontynuacja gry.	1,5 pkt
+	Systematyczna praca z GIT+	1 pkt
++	Komentarze do funkcji w kodzie	0,5 pkt
*	Zadania z (*) nie muszą być wykonywane po kolei.	